Senior Java Backend SQL Revision Lab

This document contains: Mock tables Insert statements with sample data SQL revision exercises Advanced backend SQL topics

Mock Tables + Insert Statements

```
-- ========================================
-- DEPARTMENTS
-- ========================================

CREATE TABLE departments (
    id BIGINT PRIMARY KEY,
    name VARCHAR(100)
);

INSERT INTO departments VALUES
(1, 'Engineering'),
(2, 'Finance'),
(3, 'HR'),
(4, 'Operations');

-- ========================================
-- USERS
-- ========================================

CREATE TABLE users (
    id BIGINT PRIMARY KEY,
    name VARCHAR(100),
    email VARCHAR(100),
    salary DECIMAL(10,2),
    department_id BIGINT,
    created_at TIMESTAMP
);

INSERT INTO users VALUES
(1, 'Rahul', 'rahul@test.com', 120000, 1, CURRENT_TIMESTAMP),
(2, 'John', 'john@test.com', 90000, 1, CURRENT_TIMESTAMP),
(3, 'Alice', 'alice@test.com', 150000, 2, CURRENT_TIMESTAMP),
(4, 'Bob', 'bob@test.com', 80000, 3, CURRENT_TIMESTAMP),
(5, 'David', 'david@test.com', 95000, 1, CURRENT_TIMESTAMP),
(6, 'Emma', 'emma@test.com', 110000, 4, CURRENT_TIMESTAMP),
(7, 'Sophia', 'sophia@test.com', 105000, 2, CURRENT_TIMESTAMP),
(8, 'Liam', 'liam@test.com', 75000, 3, CURRENT_TIMESTAMP);

-- ========================================
-- PRODUCTS
-- ========================================

CREATE TABLE products (
    id BIGINT PRIMARY KEY,
    name VARCHAR(100),
    category VARCHAR(50),
    price DECIMAL(10,2),
    stock_quantity INT
);

INSERT INTO products VALUES
(1, 'Laptop', 'Electronics', 85000, 10),
(2, 'Phone', 'Electronics', 50000, 25),
(3, 'Keyboard', 'Accessories', 2500, 50),
(4, 'Mouse', 'Accessories', 1500, 100),
(5, 'Monitor', 'Electronics', 18000, 20);

-- ========================================
-- ORDERS
-- ========================================

CREATE TABLE orders (
    id BIGINT PRIMARY KEY,
    user_id BIGINT,
    total_amount DECIMAL(10,2),
    status VARCHAR(50),
    created_at TIMESTAMP
);
```

```
INSERT INTO orders VALUES
(1, 1, 87000, 'COMPLETED', '2026-01-10'),
(2, 2, 50000, 'COMPLETED', '2026-01-12'),
(3, 3, 2500, 'PENDING', '2026-01-13'),
(4, 1, 18000, 'COMPLETED', '2026-01-14'),
(5, 4, 12000, 'FAILED', '2026-01-15');

-- ========================================
-- ORDER ITEMS
-- ========================================

CREATE TABLE order_items (
    id BIGINT PRIMARY KEY,
    order_id BIGINT,
    product_id BIGINT,
    quantity INT,
    item_price DECIMAL(10,2)
);

INSERT INTO order_items VALUES
(1, 1, 1, 1, 85000),
(2, 1, 3, 1, 2500),
(3, 2, 2, 1, 50000),
(4, 3, 3, 1, 2500),
(5, 4, 5, 1, 18000);

-- ========================================
-- PAYMENTS
-- ========================================

CREATE TABLE payments (
    id BIGINT PRIMARY KEY,
    order_id BIGINT,
    payment_method VARCHAR(50),
    payment_status VARCHAR(50),
    amount DECIMAL(10,2)
);

INSERT INTO payments VALUES
(1, 1, 'UPI', 'SUCCESS', 87000),
(2, 2, 'CARD', 'SUCCESS', 50000),
(3, 3, 'UPI', 'PENDING', 2500),
(4, 4, 'NETBANKING', 'SUCCESS', 18000),
(5, 5, 'CARD', 'FAILED', 12000);
```

Revision Exercises

CORE SQL
1. Highest salary employee per department
2. Users without orders
3. Products never ordered
4. Top customer by revenue
5. Monthly revenue aggregation
6. Second highest salary

WINDOW FUNCTIONS
7. Top 3 salaries per department
8. Running totals
9. Previous order amount using LAG()
10. Nth highest salary

INDEXING + OPTIMIZATION
11. Create composite indexes
12. Explain analyze queries
13. Detect full table scans
14. LIKE optimization problems
15. Covering index exercises

TRANSACTIONS + LOCKING
16. Inventory overselling prevention
17. SELECT FOR UPDATE
18. Optimistic locking using version column
19. Deadlock scenarios
20. Isolation levels

PAGINATION
21. LIMIT OFFSET pagination
22. Cursor pagination
23. Deep offset performance issue

SYSTEM DESIGN SQL
24. BookMyShow seat locking
25. Payment idempotency
26. Audit table design
27. Soft delete strategy
28. Scaling billion-row orders table